

1. [Exercici 1] (2 punts):

L'empresa Nice Buildings està especialitzada en pintar façanes de grans edificis que segueixen estrictes criteris d'impacte cromàtic i econòmic. Avui han rebut un encàrrec d'un arquitecte que vol un determinat to de pintura per a les parets i un curiós efecte cromàtic a les finestres de l'edifici que està construint. A les especificacions del projecte s'indica que les finestres són rectangulars, que totes les finestres d'una mateixa planta tenen la mateixa alçada i línia de base, i que cada finestra s'ha de pintar d'un color. A més, a les finestres que estan sobre la mateixa vertical de l'edifici (encara que sigui parcialment) no s'hi pot repetir cap color.

El problema que es troba Nice Buildings és que les finestres no segueixen cap patró regular: tot i que les finestres sí que s'arreglaren per pisos, dins d'aquests poden estar col·locades a qualsevol posició i tenir qualsevol amplada. A més, per criteris estètics i econòmics, l'empresa ha de fer servir el *mínim nombre de colors possibles* per pintar les finestres. A la figura podeu veure un exemple de façana d'un edifici en què s'han pogut complir les especificacions de l'arquitecte fent servir només dos colors.



De cada finestra i sabem la planta on es troba (p_i), i les posicions d'inici (x_e) i de final (x_d) de la finestra, mesurades per la distància a la paret de l'esquerra de la façana de l'edifici.

Dissenyeu un algorisme eficient que, donada una llista $\{(p_1, x_{e_1}, x_{d_1}), \dots, (p_n, x_{e_n}, x_{d_n})\}$ que descriu les n finestres de l'edifici, determini una forma de pintar-les respectant les restriccions de l'arquitecte i fent servir el mínim nombre de colors possibles.

Una solució:

Aquest problema és molt similar al problema 2.3 (Partició d'interval) vist a classe. En aquest problema, considerem el nombre de colors diferents necessaris per pintar les finestres (R , els nostres recursos). Volem minimitzar el nombre de recursos necessaris (R^*) respectant la restricció de l'enunciat: "a les finestres que estan sobre la mateixa vertical de l'edifici (encara que sigui parcialment) no s'hi pot repetir cap color".

Definim $I = \min\{x_{e_i}\}$, $T = \max\{x_{d_i}\}$ i $R(x)$ el nombre de finestres a la mateixa vertical del punt x , podem veure que òbviament $R^* = \max_{x \in [I, T]} R(x)$. Intuïtivament, és com si recorreguéssim les finestres ordenades, des de l'inici de la finestra situada més a la dreta fins al final de la finestra situada més a l'esquerra, i calculéssim a quin punt s'apilen més finestres (punt en què necessitem més colors). Per determinar eficientment el mínim nombre de colors necessaris R^* , seguirem els següents passos.

- Ordenarem les finestres per posició d'inici x_{e_i} .
- Recorrerem les finestres per posició d'inici (d'esquerra a dreta).
- Mantindrem un conjunt P amb les finestres que estan sobre la mateixa vertical (*finestres actives*) que necessiten estar pintades de colors diferents. Mantenim P ordenat per posició de finalització x_{d_j} . Podem usar un min-heap on x_{d_j} és la prioritat.
- Per cada finestra i que processi el nostre algoritme, retirarem de P totes aquelles finestres j tal que $x_{d_j} < x_{e_i}$, donat que ja no estaran sobre la mateixa vertical, alliberant recursos (és a dir, colors).

Podem formalitzar-ho en el següent algoritme.

```

function MINCOLORS( $W$  finestres,  $n$  nombre de finestres)
  ▷ Ordenar usant el teu algoritme preferit de cost  $O(n \log n)$ 
  ▷  $x_{e_1} \leq x_{e_2} \leq \dots \leq x_{e_n}$ 
   $W_{sorted} \leftarrow \text{MERGESORT}(W)$ 
   $P \leftarrow \emptyset$  ▷ Min-heap amb prioritat  $x_{d_j}$ 
   $R \leftarrow 0$ ;  $R^* \leftarrow 0$  ▷ Inicialitza a zero
  for  $i \leftarrow 1$  to  $n$  do
     $\langle x_{e_i}, x_{d_i} \rangle \leftarrow W_{sorted}[i]$ 
    ▷ Extreure de  $P$  les finestres que ja no estan sobre la vertical
    while  $P \neq \emptyset$  do
       $\langle j, x_{d_j} \rangle \leftarrow P.\text{top}()$ 
      ▷ Verifica si les finestres a  $P$  segueixen sobre la mateixa vertical (finestres actives)
      if  $x_{d_j} \geq x_{e_i}$  then break
      ▷ Reduir el nombre de colors ocupats
       $R = R - 1$ 
       $P.\text{pop}()$ 
    end while
    ▷ Introduir la nova finestra a  $P$  i augmentar colors requerits
     $P.\text{push}(\langle i, x_{d_i} \rangle)$ 
     $R = R + 1$ 
     $R = \max\{R^*, R\}$ 
  end for
  return  $R^*$ 
end function

```

Respecte al cost de l'algoritme, ordenar les finestres té un cost $O(n \log n)$ (fent-ho amb un algoritme d'ordenació eficient), el bucle **for** realitza n iteracions i cada operació amb el min-heap té un cost $O(\log n)$ (on les n finestres entraran i sortiran del min-heap). Per tant, el cost global de l'algoritme serà $O(n \log n)$.

2. [Exercici 2] (2,5 punts):

Tenim una expressió aritmètica que conté n nombres reals i $n - 1$ operadors, cadascun $+$ o $*$. L'expressió no està parentitzada i no sabem en quin ordre hi hem d'aplicar les operacions. El que sí que sabem, però, és que volem fer les operacions en un ordre que maximitzi el valor de l'expressió resultant. És a dir, volem inserir adequadament els parèntesis a l'expressió per tal que el seu valor en avaluar-la sigui maximitzat. Per exemple:

- Per a l'expressió $6 * 3 + 2 * 5$, l'ordre òptim és sumar primer els nombres del mig i després realitzar les multiplicacions: $(6 * (3 + 2)) * 5 = 150$.
- Per a l'expressió $0, 1 * 0, 1 + 0, 1$, l'ordenació òptima és realitzar primer la multiplicació i després la suma: $(0, 1 * 0, 1) + 0, 1 = 0, 11$.
- Per a l'expressió $(-3) * 3 + 3$, l'ordenació òptima és $((-3) * 3) + 3 = -6$.

Doneu un algorisme, tan eficient com pugueu, per tal de calcular quin és el valor màxim obtenible en parentitzar l'expressió d'entrada i quina és exactament aquesta parentització.

Una solució:

Aquest problema és semblant al problema de la parentització de la multiplicació de matrius, vist a les classes de teoria. En aquest cas hem de decidir com parentitzar la seqüència d'entrada per tal de maximitzar l'expressió donada. Resoldrem el problema mitjançant programació dinàmica perquè, com veurem, el problema té les dues propietats requerides: subestructura òptima i solapament de subproblemes.

Per a plantejar una resolució per programació dinàmica necessitem primer definir la notació necessària. Denotarem els nombres de l'expressió com $a_1, a_2, \dots, a_n$, i els operadors com $op_1, op_2, \dots, op_{n-1}$, per tant, una expressió donada és de la forma

$$a_1 \ op_1 \ a_2 \ \dots \ a_{n-1} \ op_{n-1} \ a_n.$$

Sigui $M(i, j)$ el **màxim** valor que es pot obtenir de la subexpressió que comença a a_i i que acaba a a_j (és a dir, la subexpressió $a_i \ op_i \ \dots \ op_{j-1} \ a_j$), i sigui $m(i, j)$ el **mínim** valor que es pot obtenir en avaluar la subexpressió que comença a a_i i acaba a a_j .

Tota expressió donada acabarà operant dues subexpressions (parentitzades) mitjançant un darrer operador, és a dir, $((expr_1) \ op_k \ (expr_2))$. Per tal que el resultat d'aquesta darrera avaluació sigui òptima (màxima), cal que l'avaluació de les dues subexpressions que s'operen també ho siguin. Però observeu que hem de mantenir tant el màxim com el mínim de tota subexpressió, perquè el valor màxim de l'avaluació global podria provenir de multiplicar dos subexpressions que avaluen a números negatius.¹

Basant-nos en aquesta subestructura òptima observada, plantegem la recurrència que expressa la solució a un subproblema general en relació a solucions de subproblemes més petits. Per avaluar una subexpressió $a_i \ \dots \ a_j$, podem separar-la en dos problemes al k -èsim operador

¹Si els nombres fossin naturals (i no reals, com diu l'enunciat), aleshores n'hi hauria prou amb mantenir els valors màxims i el problema seria més senzill.

i, recursivament, resoldre les subexpressions $a_i \dots a_k$ i $a_{k+1} \dots a_j$. En fer això hem de considerar totes de combinacions de maximització i minimització dels subproblemes.

Els casos base són $M(i, i) = m(i, i) = a_i, \forall i : 1 \leq i \leq n$.

Els casos recursius són:

$$M(i, j) = \max_{i \leq k < j} \left\{ \begin{array}{l} \max \left(\begin{array}{l} M(i, k) \text{ op}_k M(k+1, j), \\ M(i, k) \text{ op}_k m(k+1, j), \\ m(i, k) \text{ op}_k M(k+1, j), \\ m(i, k) \text{ op}_k m(k+1, j) \end{array} \right) \end{array} \right\}$$

$$m(i, j) = \min_{i \leq k < j} \left\{ \begin{array}{l} \min \left(\begin{array}{l} M(i, k) \text{ op}_k M(k+1, j), \\ M(i, k) \text{ op}_k m(k+1, j), \\ m(i, k) \text{ op}_k M(k+1, j), \\ m(i, k) \text{ op}_k m(k+1, j) \end{array} \right) \end{array} \right\}$$

El valor $M(1, n)$ d'aquesta recurrència és l'objectiu del problema; és la solució que estem buscant.

Hi ha un total de $O(n^2)$ subproblemes: dos per cadascun dels parells d'índexos $1 \leq i \leq j \leq n$. Els subproblemes $M(i, j)$ i $m(i, j)$ han de considerar $O(j - i) = O(n)$ subproblemes més petits. El temps total de l'algorisme és, doncs, $O(n^3)$.

El terme $M(1, n)$ només indicarà el valor numèric de l'avaluació de la parentització maximitzadora de tota l'expressió d'entrada. Per tal de determinar on s'han de col·locar exactament els parèntesis necessitem guardar adicionalment, per a cada $M(i, j)$ i $m(i, j)$, quina k (punt de tall) produeix el màxim i mínim, respectivament. Amb aquesta informació, una vegada haguem calculat $M(1, n)$, podrem reconstruir en temps $O(n)$ la parentització.

3. [Exercici 3] (2,5 punts):

L'empresa DistFast està interessada a trobar un mecanisme per poder suggerir als seus conductors rutes per fer la distribució de material dintre del centre de Barcelona tan ràpid com sigui possible. En aquests moments DistFast té un model simplificat de Barcelona, que està format per un graf $G = (V, E)$ on els vèrtexs representen cruïlles, i les arestes representen trams de carrer que connecten les cruïlles dels seus extrems. Al mapa tenen marcades un conjunt de cruïlles $B \subseteq V$, que són les més properes als punts on tenen els magatzems d'on ha de sortir el material, i també un altre conjunt $C \subseteq V$, a on s'hi ha de distribuir el material (podeu suposar que $B \cap C = \emptyset$).

Durant el dia van seguint les informacions del trànsit dins de la ciutat i això els permet fer una classificació dels trams de carrers segons tres nivells de saturació de trànsit: alt, mitjà o baix. També determinen amb aquesta informació el subconjunt $D \subseteq V$ de cruïlles que voldrien evitar.

Les rutes són camins que comencen a alguna cruïlla de B i que acaben a alguna de les cruïlles de C . DistFast està interessada a trobar el *màxim nombre possible de rutes* que podrien fer servir els seus conductors, d'acord amb dos criteris diferents:

- (a) Els camins no comparteixen vèrtexs (cruïlles) entre ells.
- (b) Un tram de carrer molt saturat només es pot fer servir com a molt en una ruta, un de saturació mitjana en, com a molt, 10 i un de baix nivell de saturació no té restriccions. Una cruïlla a D només pot aparèixer en, com a molt, dues rutes.

Doneu algorismes tan eficients com pugueu que, donats G, B, C, D i una avaluació del nivell de saturació del trànsit, determini un conjunt de rutes de grandària màxima d'acord amb cadascun dels dos criteris.

Una solució: Aquest exercici és bàsicament igual al problema 4.16 (RACC). Essencialment, busquem el màxim nombre de rutes a un graf amb certes restriccions. Per a l'**apartat (a)**, resumirem les restriccions del nostre problema per després modelar-les amb una xarxa s-t.

- R1: $G = (V, E)$ és un mapa amb vèrtexs V (cruïlles) i arestes E (carrers).
- R2: $B \subseteq V$ són els vèrtexs d'inici dels camins.
- R3: $C \subseteq V$ són els vèrtexs de final dels camins.
- R4: $D \subseteq V$ són els vèrtexs que no estan continguts en els camins.
- R5: Els camins han de ser de vèrtexs disjunts.

Procedim a modelar i formalitzar el problema i les seves restriccions usant una xarxa s-t $\mathcal{N}$.

- **(R1)**, Com que $G = (V, E)$ és un graf no dirigit, el transformarem en un graf dirigit $G' = (V, E')$ on, per cada aresta $e = (v_1, v_2) \in E$, ara tindrem dos arcs $e_1 = (v_1, v_2)$ i $e_2 = (v_2, v_1)$ ($e_1, e_2 \in E'$), un d'anada i un altre de tornada.
- **(R2)**, connectarem tots els vèrtexs $v \in B$ amb la font s ($\{(s, v) \mid v \in B\}$).
- **(R3)**, connectarem tots els vèrtexs $u \in C$ amb l'embornal (*sink*) t ($\{(u, t) \mid u \in C\}$).
- **(R4)**, eliminarem tot arc que connecti $w \in D$ ($\{(w, x), (x, w) \mid w \in D \wedge x \in V\}$).

- **(R5)**, per limitar la capacitat de les cruïlles (vèrtex disjunts), desdoblaem tots els vèrtexs $v \in V$ en dos (v_{in} i v_{out}). v_{in} rebrà els arcs d'entrada i de v_{out} en sortiran els de sortida. L'arc connectant les dues còpies (v_{in}, v_{out}) tindrà capacitat 1. La resta d'arcs tindrà capacitat ∞ .

Un camí de s a t té la forma $s \rightarrow v \rightarrow \dots \rightarrow u \rightarrow t$, si transporta una unitat de flux interpretarem que hi ha una ruta des de $v \in B$ fins a $u \in C$ que compleix amb totes les restriccions. Pel teorema de Menger, el nombre màxim de camins disjunts $s \rightsquigarrow t$ és igual al nombre d'arcs mínim que travessa un tall s - t ; és a dir, el valor del flux màxim en $\mathcal{N}$. Per això, per resoldre el problema calcularem un flux amb valor màxim a la xarxa $\mathcal{N}$. Després, en podem extreure les rutes utilitzant el graf format per les arestes amb flux 1. Amb un BFS podem extreure'n un camí, restar una unitat de flux a les arestes d'aquest camí, i repetim el procés fins a extreure'n els f^* camins. Proposo el següent algorisme.

```

function DISTFAST( $G, B, C, D$ )
  ▷ Construir la xarxa s-t
   $\mathcal{N} \leftarrow \text{ConstruirXarxa}(G, B, C, D)$ 
  ▷ Computar el flux màxim
   $F \leftarrow \text{MaxFlow}(\mathcal{N})$ 
  ▷ Obtenir rutes
   $Rutes \leftarrow \text{ObtenirRutes}(\mathcal{N}, F)$ 
end function

```

Amb n (el nombre inicial de vèrtexs) i m (el nombre inicial d'arestes), podem veure que cap transformació aplicada en el modelat incrementa n i m excepte en factors constants. Construir la xarxa s - t costa $O(n + m)$. Calcular el flux màxim usant Ford-Fulkerson té cost pseudopolinòmic $O(f^*(m + n))$, on no pot haver-hi més camins disjunts que cruïlles ($f^* \leq n$), per tant $O(n(m + n))$. Noteu que, si decidim utilitzar Edmonds-Karp, tindrem un cost $O(mn(m + n))$ que no sembla millorar la fita de FF. Finalment, extreure'n n camins com a màxim tindrà cost $O(n(m + n))$. En definitiva, el nostre algorisme té cost $O(n(m + n))$ en el pitjor cas.

A l' apartat **(b)**, s'apliquen les restriccions **{R1, R2, R3}** i afegim les següents.

- R6: Arestes amb saturació alta, $a \in S_{alta} \subseteq E$, només poden pertànyer a 1 ruta.
- R7: Arestes amb saturació mitjana, $m \in S_{mitjana} \subseteq E$, només poden pertànyer a 10 rutes.
- R8: Arestes amb saturació baixa, $b \in S_{baixa} \subseteq E$, poden pertànyer a ∞ rutes.
- R9: $D \subseteq V$ vèrtexs poden pertànyer a 2 rutes.

En aquest cas **{R6,R7,R8}**, només hem d'ajustar la capacitat de les arestes en S_{alta} , $S_{mitjana}$ i S_{baixa} als valors 1, 10 i ∞ , respectivament. A més **(R9)**, només desdoblaem els vèrtexs en D i l'arc connectant les dues còpies tindrà capacitat 2. La resta de la resolució és idèntica a l'apartat (a).

4. [Exercici 4] (3 punts: 0,75 per qüestió):

- (a) Siguin A i B dos conjunts, ambdós amb n elements, tal que els elements de A es troben a l'ordinador P i els de B al Q . P i Q es poden comunicar entre ells enviant-se missatges i poden executar localment qualsevol tipus d'operacions. Per simplificar-ho, podeu suposar que tots els elements són diferents.

Volem un algorisme per trobar l' n -èsim element més petit d' $A \cup B$. Per fer-ne l'anàlisi temporal volem comptabilitzar només el nombre de missatges, assumint que un missatge pot contenir o bé un número entre 1 i n , o bé un element del conjunt.

Es pot trobar l' n -èsim element més petit d' $A \cup B$ amb $o(n)$ missatges en cas pitjor? Justifiqueu-ne la vostra resposta.

Una resposta curta i vàlida

És un problema idèntic al problema 1.20 vist a classe. Donats $2n$ elements en A i B , el n -èsim element més petit de $A \cup B$ és la mediana. Per obtenir un cost $o(n)$, necessitem un algorisme que redueixi la mida del problema a una fracció del mateix en cada pas. Primer, ordenem localment cada conjunt A i B (no afecta el cost de pas de missatges). Siguin m_a i m_b les medianes de A i B , wlog, si $m_a < m_b$, la mediana cercada està entre m_a i m_b . Podem descartar $n/2$ valors de cada conjunt i procedir recursivament en les meitats restants. L'algorisme acaba quan queden dos elements (el menor d'ells és la mediana). L'algorisme realitza $O(\log n)$ passos (missatges entre ordinadors), on $O(\log n) \subset o(n)$.

Una resposta més llarga (amb un fi docent)

Per a poder obtenir cost $o(n)$ necessitem tenir un algorisme que a cada pas redueixi la mida del problema a una fracció de la mida original; és a dir, $O(\log n) \subset o(n)$. Primer, podem ordenar localment cada conjunt A i B , ja que no afecta el cost $O(\log n)$ de pas de missatges. Amb A i B ordenats, sigui $m_a = A[n/2]$ i $m_b = B[n/2]$ (la mediana de cada conjunt). Com que no hi ha elements repetits, un dels valors és més gran que l'altre. Sense pèrdua de generalitat, suposem que $m_a < m_b$. Hi ha com a mínim n elements menors que m_b i com a mínim n elements més grans que m_a , així que la mediana que busquem compleix $m_a \leq m \leq m_b$. Per això, els primers $n/2$ elements de A són menors o iguals que m_a (i que m) i els últims $n/2$ elements de B són més grans o iguals que m_b (i més grans que m). En eliminar el mateix nombre de valors més grans que menors, la mediana del conjunt de valors que resten continua sent m . A més, eliminem el mateix nombre $n/2$ de valors de cada conjunt.

procedure MEDIANA(a_1, a_2, b_1, b_2)

▷ Cas base

if $a_2 - a_1 = 1$ **return** $\min(A[a_1], B[b_2])$

▷ Cas general recursiu

$r \leftarrow (a_2 - a_1)/2$

$(m_a, m_b) \leftarrow (A[a_1 + r], B[b_1 + r])$

if $m_a < m_b$ **then**

$(a_1, b_2) \leftarrow (a_1 + r, b_2 - r)$

else

```

         $(b_1, a_2) \leftarrow (b_1 + r, a_2 - r)$ 
    end if
    return MEDIANA( $a_1, a_2, b_1, b_2$ )
end procedure

```

L'algoritme mantindrà en tot moment els dos trossos de cada conjunt dels quals hem d'obtenir la mediana i finalitzarà quan tinguem dos trossos amb un únic element. En aquest cas base, el menor dels dos elements és la mediana. La crida **Mediana**(a_1, a_2, b_1, b_2) obtindrà la mediana de $\{A[i] \mid a_1 \leq i < a_2\} \cup \{B[j] \mid b_1 \leq j < b_2\}$. Per resoldre el problema serà suficient executar **Mediana**(0, n , 0, n). Al llarg de l'algoritme mantindrem la propietat $a_2 - a_1 = b_2 - b_1$, així garantim que el tros d' A i el tros de B a tractar tenen la mateixa mida. L'algoritme realitza $O(\log n)$ passos (missatges entre ordinadors), on $O(\log n) \subset o(n)$.

- (b) És cert que l'algorisme de Prim per a trobar l'MST d'un graf $G = (V, E)$ amb pesos $w(e)$ per cada aresta a E és correcte únicament quan $w(e) \geq 0$, per tota $e \in E$?

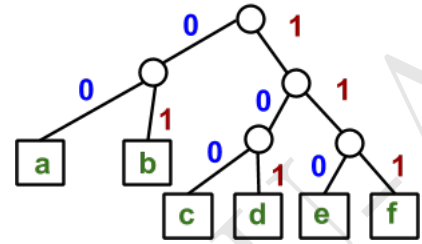
Una resposta curta i vàlida: Rotundament fals. L'algoritme de Prim és correcte per trobar el MST en un graf amb pesos negatius $w(e) < 0$ i positius $w(e) \geq 0$.

Una resposta més llarga (amb un fi docent): Rotundament fals. L'algoritme de Prim és correcte per trobar el MST en un graf amb pesos negatius $w(e) < 0$ i positius $w(e) \geq 0$. L'algoritme de Jarník-Prim manté un conjunt de vèrtexs $Vistos \subset V(G)$ i, en cada etapa de l'algoritme, selecciona l'aresta de menor pes (que pot ser negatiu, $w(e) < 0$) que uneix a un vèrtex u de $Vistos$ amb un vèrtex v no vist (és a dir, en $V(G) - Vistos$). Aquesta aresta no pot crear un cicle i s'afegeix al conjunt A . Així mateix el vèrtex v passa a ser de $Vistos$. La correcció de l'algoritme és immediata. El conjunt A respecta el tall $\langle Vistos, V(G) - Vistos \rangle$ i seleccionem l'aresta de menor pes que creua el tall per afegir-la a A .

- (c) Ens és donat un fitxer de text que conté només caràcters del conjunt $\{a, b, c, d, e, f\}$. Sabem, a més, que la freqüència de cada lletra dins del fitxer és:

a: 5%	b: 5%	c: 10%
d: 15%	e: 25%	f: 40%

Considerem l'arbre de la figura per a un codi prefix format pels caràcters $\{a, b, c, d, e, f\}$.



Responen de manera raonada a les següents preguntes:

- Quin és el nombre mitjà de bits per caràcter utilitzat per codificar el fitxer?
- Hi ha un arbre de prefixos per al conjunt $\{a, b, c, d, e, f\}$ que resultaria en menys bits per caràcter de mitjana per a les freqüències donades dels caràcters?
- Coneixes algun codi prefix que es faci servir actualment?

Resposta:

- (a) Segons l'arbre donat, la codificació dels caràcters és la següent:

a $\rightarrow$ 00; **b** $\rightarrow$ 01; **c** $\rightarrow$ 100; **d** $\rightarrow$ 101; **e** $\rightarrow$ 110; **f** $\rightarrow$ 111

Per tant, els caràcters **a** i **b** utilitzen dos bits, i els caràcters **d**, **e** i **f** n'utilitzen tres. Aleshores el nombre mitjà de bits per caràcter utilitzat per codificar el fitxer és: $(0.05 + 0.05) \cdot 2 + (0.10 + 0.15 + 0.25 + 0.40) \cdot 3 = 2.9$

- (b) Sí, per exemple l'arbre de Huffman. Aplicat a les freqüències donades es codificaria:

a $\rightarrow$ 11111; **b** $\rightarrow$ 11110; **c** $\rightarrow$ 1110; **d** $\rightarrow$ 110; **e** $\rightarrow$ 10; **f** $\rightarrow$ 0

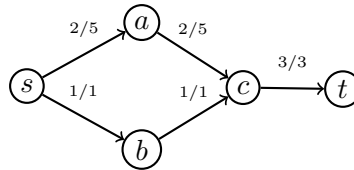
Per tant, els caràcters **a** i **b** utilitzarien cinc bits, el caràcter **c** n'utilitzaria quatre, el caràcter **d** tres, el caràcter **e** en faria servir dos, i el caràcter **f** només un. Aleshores el nombre mitjà de bits per caràcter utilitzat per codificar el fitxer seria: $(0.05 + 0.05) \cdot 5 + (0.10 \cdot 4) + (0.15 \cdot 3) + (0.25 \cdot 2) + (0.40 \cdot 1) = 2.25$

- (c) Els prefixos telefònics, els codis postals, el codi Morse (amb espais), els números romans, la codificació UTF-8, la notació polaca inversa, els ISBNs, etc.

(d) Digueu si és **cert** o **fals** i justifiqueu-ne la vostra resposta.

Sigui $\mathcal{N} = (V, E, c, s, t)$ una xarxa de flux amb capacitats als enters. Ens donen un flux f amb valor màxim a $\mathcal{N}$ i una aresta $e \in E$ amb $c(e) > 1$ y $f(e) > 1$. Sigui $\mathcal{N}'$ la xarxa de flux obtinguda reduint en 1 unitat la capacitat d' e . Llavors, el valor d'un flux amb valor màxim a $\mathcal{N}'$ és $\leq |f| - 1$.

Una resposta: L'afirmació és falsa. La següent xarxa s-t $\mathcal{N}$ i flux màxim f^* proporcionen un contraexemple.



Podem veure com f^* és màxim en $\mathcal{N}$ i que $e = (s, a)$ compleix que $c(e) > 1$ i $f(e) > 1$. Podem reduir en 1 unitat la capacitat de e . No obstant, això no canviarà el resultat f^* . Noteu que l'arc $d = (c, t)$ ja està saturat i un tall s-t $\langle V, T \rangle = \langle \{s, a, b, c\}, \{t\} \rangle$ demostra que el flux màxim f^* no pot ser superior a $c(V, T) = 3$.